Chirp! Project Report

ITU BDSA 2025 Group 17

Peter Dahl Hæstrup phae@itu.dk
Athena Winther atwi@itu.dk Rasmus Bondo rabh@itu.dk
Ditte Lobo dsab@itu.dk Nikolaj Schiang nsee@itu.dk

1 Design and Architecture of *Chirp!*

1.1 Domain model

The Chirp domain model consists of four entities:

1. Author (user extending ASP.NET Identity), this represents a user of the application.
2. Cheep a 160-character message with a timestamp, which an author can create and post on the Chirp social platform.
3. Follow enables authors to follow eachother and see followed users cheeps on their own timeline.
4. SavedCheep are Cheeps saved by the user.

The model implements a blogging platform with social features including following and timeline feeds. Repositiitory interfaces (ICheepRepository, IAuthorRepository) provide data access abstraction with support for pagination, search and deletion.

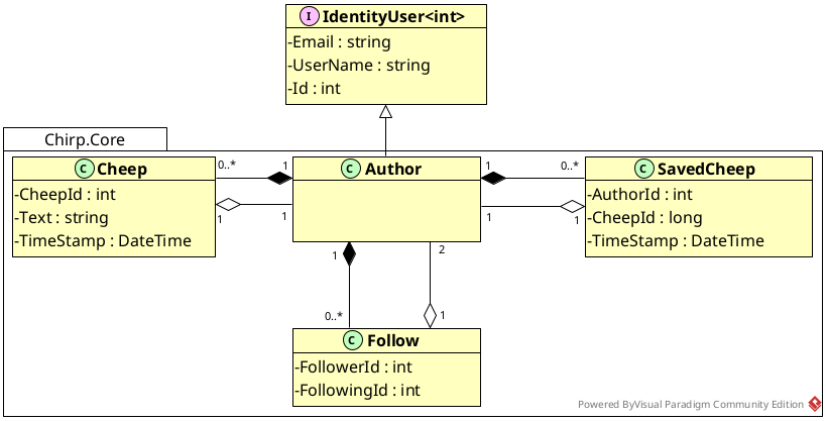

Figure 1: Illustration of the *Chirp!* data model as UML class diagram.

1.2 Architecture — In the small

The diagram shown below illustrates the program's onion architecture. The application generally follows the onion structure even though some layers are represented by more than one .NET project. The Chirp.Core .NET project is the core onion layer, on top of that is the Chirp.Repositories .NET project layer. Here the DTO's exist as they define the data contracts used across the repository, services and representation layers. Ontop of the repositories layer is the Chirp.Services .Net project layer, the service and repository layers are located withing a shared folder called Chirp.Infrastructures. The outermost layer contains the frontend Razor Pages, called Chirp.Web, and the application tests.

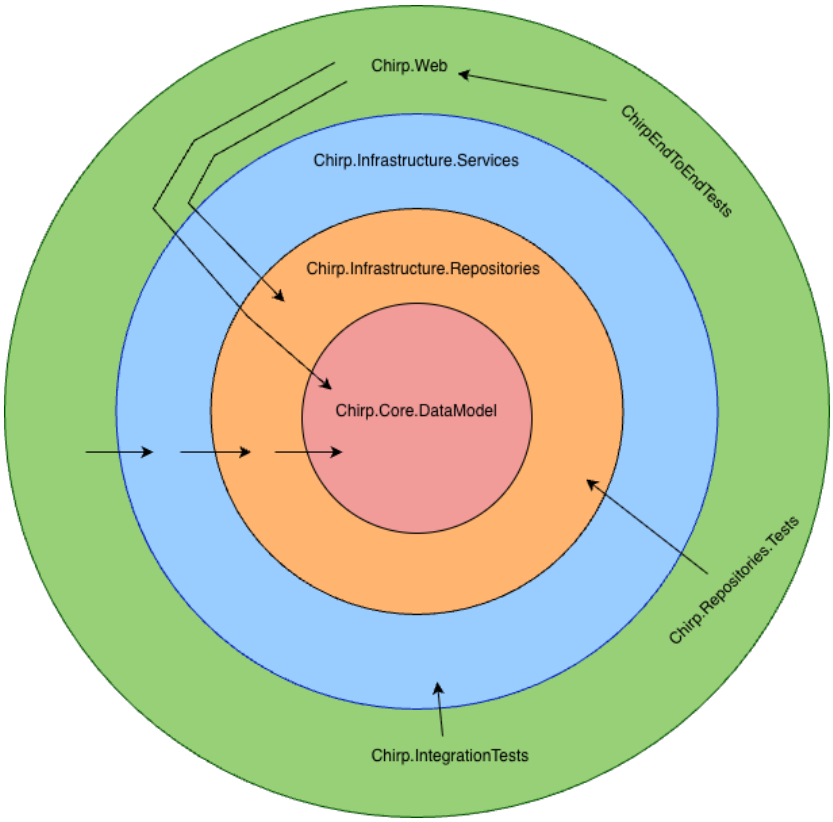

Figure 2: Onion Architecture

1.2.1 Chirp.Web

- ASP.NET Core Razor Pages
- Controllers and page models
- HTTP concerns and routing
- User interface (HTML/CSS)
- Depends on all inner layers

The reason for Chirp.Web depending on Chirp.Core is that ASP.NET Core Identity Pages require access to Author to function.

1.2.2 Chirp.Infrastructure.Services

- Service implementations: CheepService, AuthorService
- Defines use-cases for the database operations from repositories
- Workflows for database operations
- Depends on Chirp.Infrastructure.Repositories

1.2.3 Chirp.Infrastructure.Repositories

Repository Layer / Data Access

- Contains CheepRepository, AuthorRepository implementations
- Database context (CheepDBContext)
- DTO: CheepDTO, AuthorDTO
- Defines small individual operations on the database
- Depends on Chirp.Core

1.2.4 Chirp.Core

DataModel / Domain Layer (Core)

- Contains domain entities: Author, Cheep, Follow, SavedCheep
- Pure domain logic, no dependencies in Chirp
- The innermost layer with business rules

1.3 Architecture of deployed application

The Chirp application is hosted on Azure App Service. Users interact with the system through the Chirp.Web project, which provides the user interface using ASP.NET Core Razor Pages. All client interaction happens over HTTPS. When a user performs an action in the UI, Chirp.Web delegates the requests to the service layer in Chirp.Infrastructure.Services, where the possible database operations are implemented. The Service layer then calls the repository layer in Chirp.Infrastructure.Repositories to retrieve or modify data. Data persistence are handled via Entity Framework Core, which communicates with an SQLite database through the CheepDbContext, handled by ASP.NET Core Identity.

Autentication is handled in two ways: users can either register with ASP.NET Core Identity using an email, username and password and log in locally using with username and password after they have confirmed their account, or authenticate via GitHub OAuth - here GitHub manages the OAuth flow and returns authentication tokens to Chirp.Web.

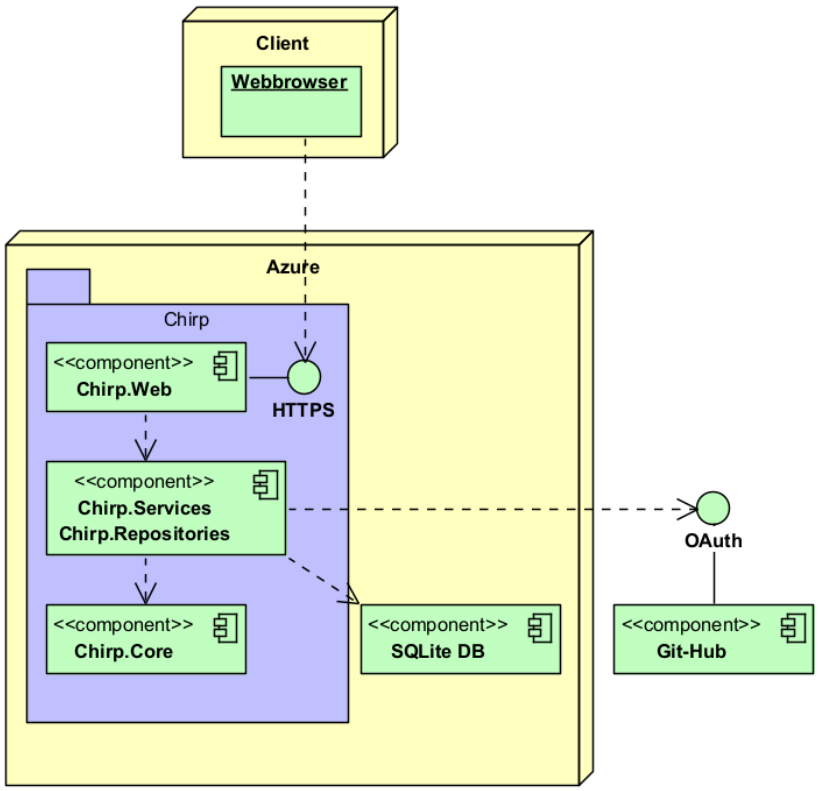

Figure 3: Deployed Components

1.4 User activities

When a user visits the application, they start on the public timeline, where they can browse cheeps, navigate between pages, search for content and view other user's timelines by clicking on the author name. From the public timeline, the user can choose to register or log-in. Registration and login can be performed either locally or via GitHub, where OAuth handles autentication and account creation externally.

If the user encounters an issue during registration or login, they are redirected back to the relevant page to retry. Once authentication succeeds, the user becomes logged in and gains access to additional features. Logged-in users can post new cheeps, follow other users and save cheeps. They can also access their personal information through the "About Me" section, where they have the option to delete their account and all associated data using the "Forget Me" functionality.

1.4.1 Non-authorized user

An unauthortized user has limited access to Chirp!'s functionality. They can view all cheeps under the public timeline, and can search for cheeps containing a specefic substring. They can also click on a users name and view that persons cheeps.

To get authorized a user must register an account if they haven't already and then log in.

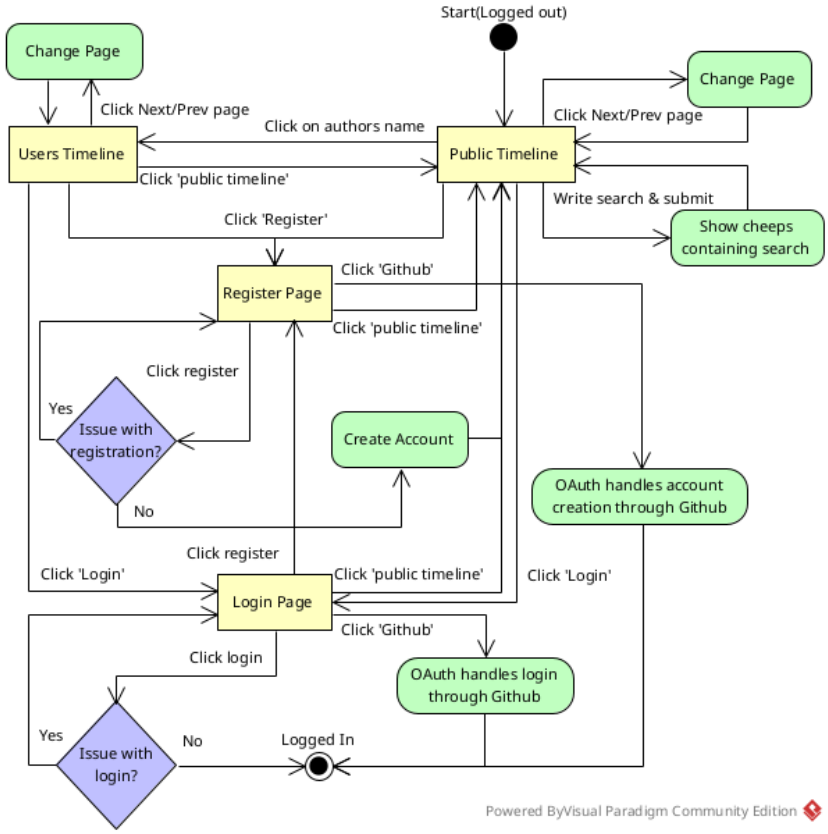

Figure 4: User that is not logged in workflow

1.4.2 Authenticated user

Authenticated users have access to the same pages as unauthorized users, with some additional pages only for the authenticated. 'My timeline' is one of these pages. On here users can view their own cheeps, as well as the cheeps from users that they follow.

Cheeps can be saved and then viewed under the 'saved' page. The cheeps are

ordered by time of saving.

The user can view who they are following under the ‘following’ page. Here they can also choose to unfollow users.

‘About me’ can be accessed to view information about the users account, such as email or username. It is also here that the user can delete their account.

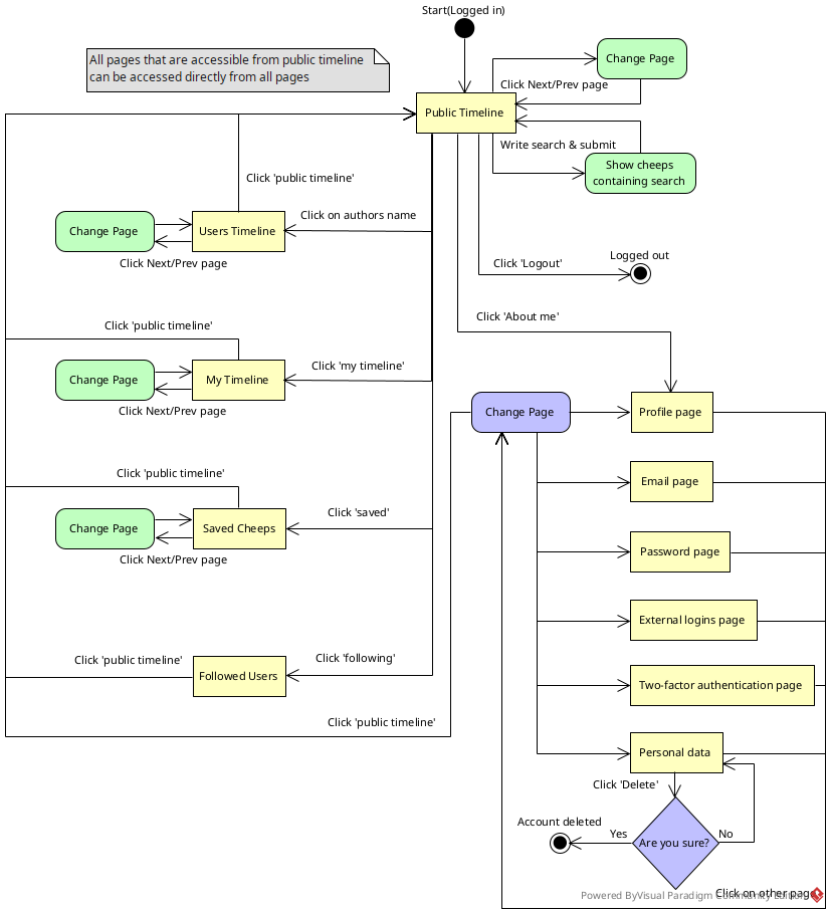

Figure 5: Logged in user workflow

1.5 Sequence of functionality/calls trough *Chirp!*

1.5.1 Register sequence

To register a user must give an email, username and password. The email and username must be unique from other users. Accounts are handled by ASP.NET Core Identity.

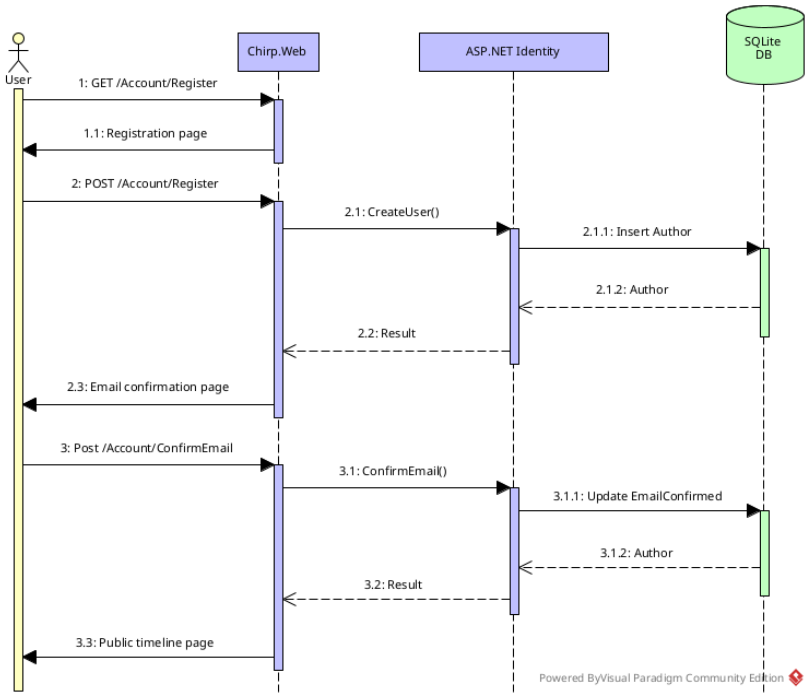

Figure 6: Register sequence

1.5.2 Authentication with Git-Hub

If the user does not wish to create an account using an email address, then they can choose to register/login with Github through OAuth. When choosing this option the user does not have to provide email, username or password, as all needed information is given by Github.

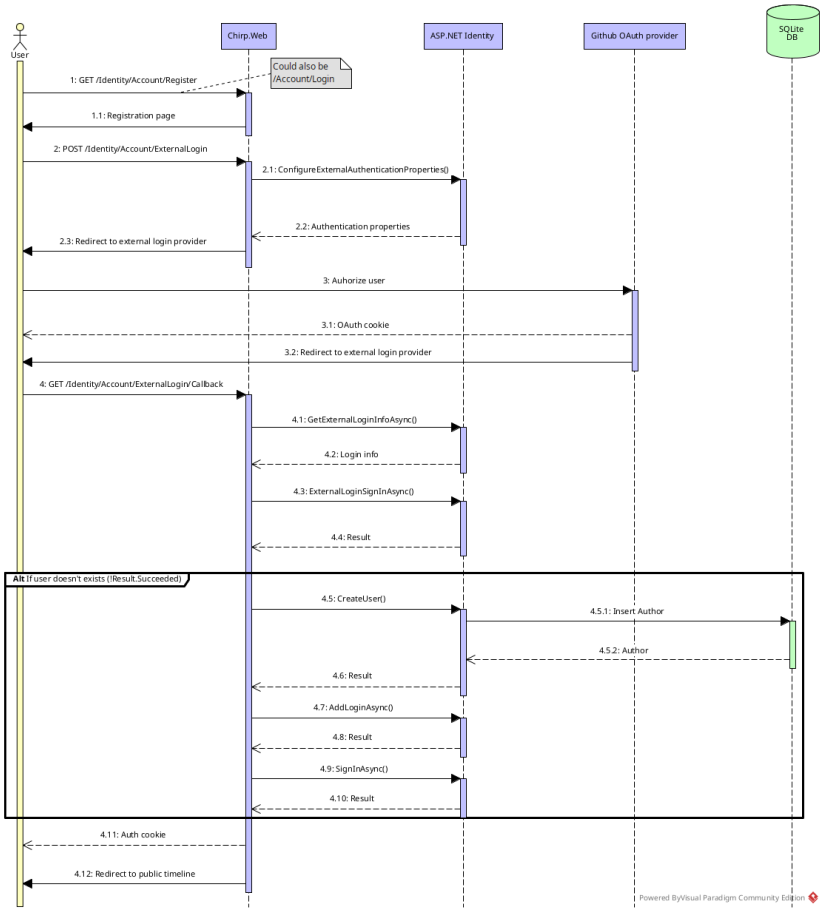

Figure 7: Authorizing with Github

1.5.3 Login sequence

Once an account is created the user must log in. This is done using the selected username and password. If using Github then this step is replaced by Github authentication through OAuth.

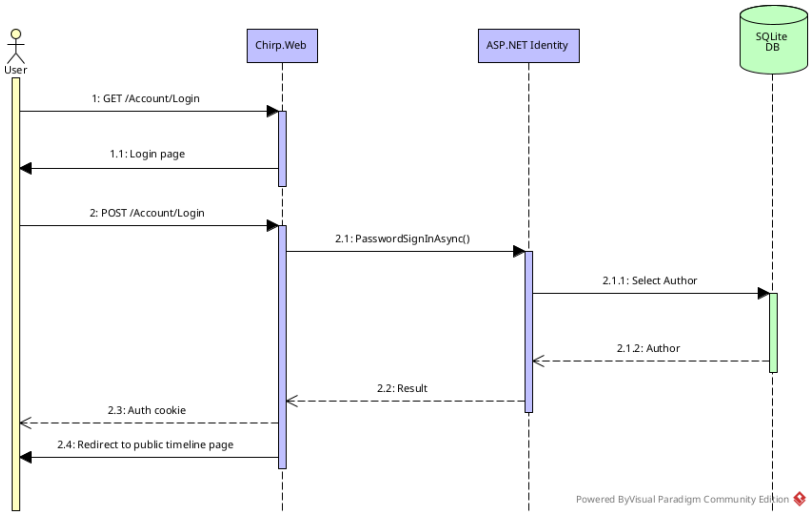

Figure 8: Login sequence

1.5.4 Non-authorized user reading the public timeline

An unauthorized user starts by sending an HTTP GET/ request to Chirp.Web. The request invokes the public page handler. The Web layer delegates the request to the service layer, which retrieves public cheeps through the repository layer. The repository queries the SQLite database via Entity Framework Core and returns the most recent cheeps as DTO's (ordered by the timestamps). These are passed back through the service to the web layer, where Razor Pages renders the HTML response. The fully rendered public timeline pages is then returned to the user.

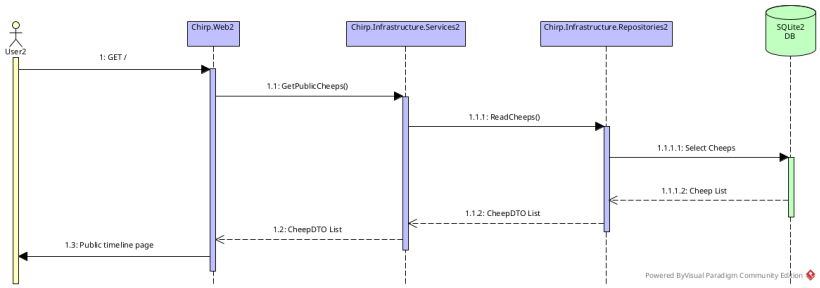

Figure 9: Getting public timeline sequence

1.5.5 Authorized user posting a cheep

An authorized user submits a new cheep by sending an HTTP POST request. The request is handled by Chirp.Web which extracts the authenticated username from the user's identity. The Web layer calls the service layer to create a new cheep for the user. The service resolves the author via the repository and then persists the new cheep through the cheeps repository using Entity Framework Core. After the database transaction succeeds, the user is redirected back to the public timeline, where the newly created cheep is now visible.

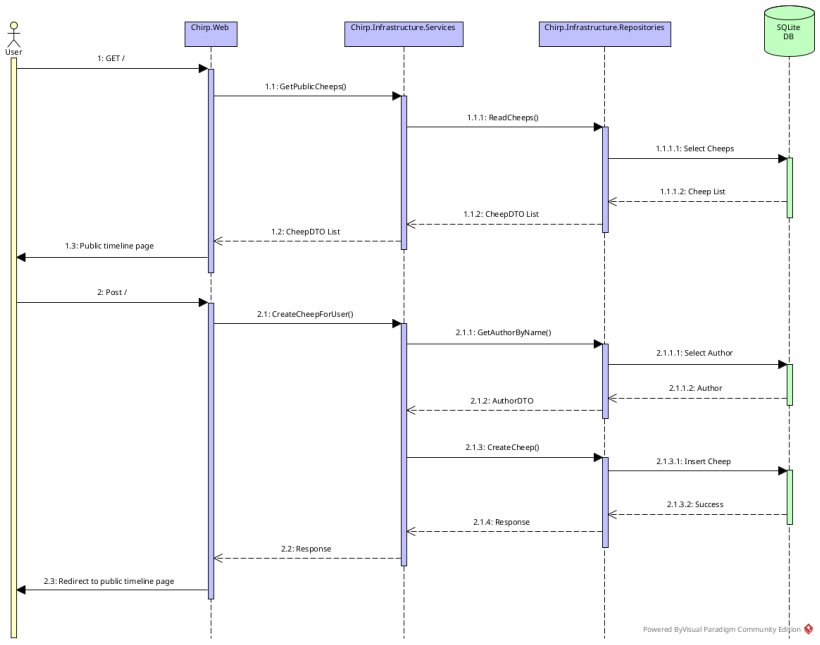

Figure 10: Posting a cheep sequence

2 Process

2.1 Build, test, release and deployment

2.1.1 Build

When the code is pushed to the main branch, the CI pipeline checks out the repository, sets up the correct .NET SDK, restores dependencies and build the application. This ensures the code compiles correctly in a clean environment.

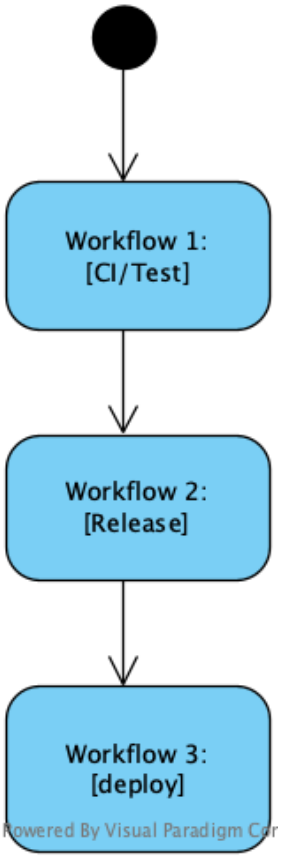

Figure 11: Workflow sequence

2.1.2 Test

After a successful build, automated tests are executed. This includes unit tests, integration tests and end-to-end tests which verify that the repositories and application logic behave as excepted. The goal is to catch errors before code is merged or released. This is done by the workflow called main.yml.

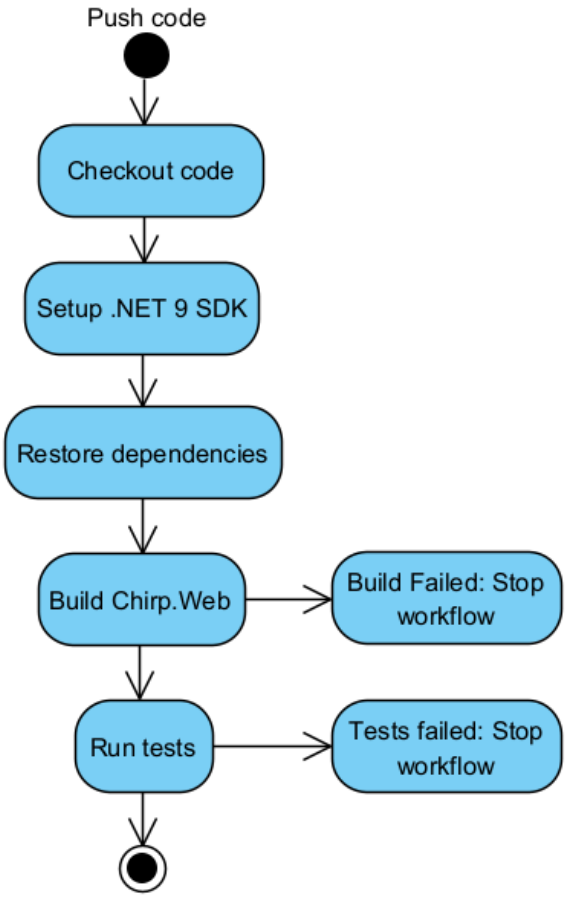

Figure 12: Build and Test workflow

2.1.3 Release

Once the build and test succeed, a release workflow packages the application in Release mode. The app is published as a self-contained, single-file executable, versioned with a tag and uploaded as a GitHub Release artifact. This is handled by the workflow release.yml.

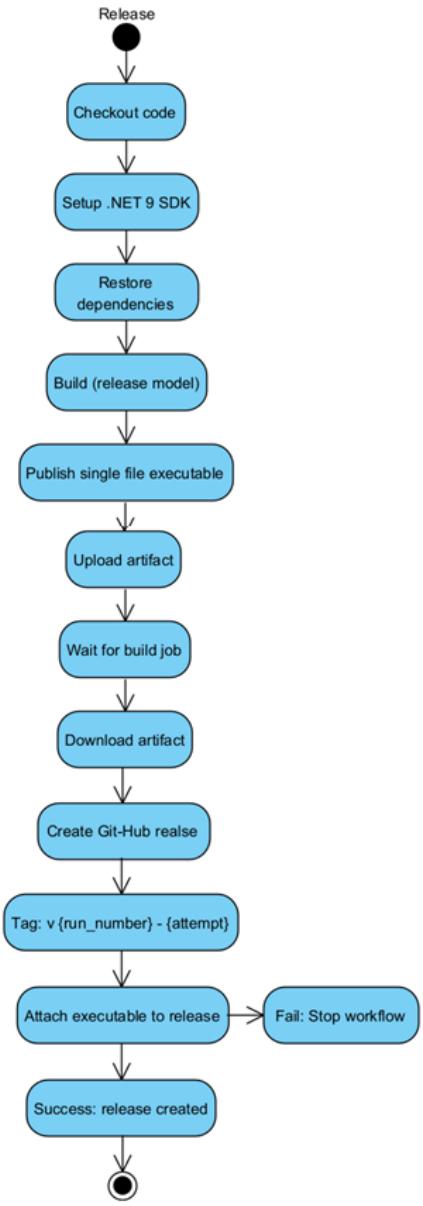

Figure 13: Release workflow

2.1.4 Deployment

In the final stage, the deployment workflow publishes the application to Azure App Service. It authenticates securely with Azure, uploads the built artifact and deploys it to the production environment. This keeps the live application automatically synchronized with the main branch. Deployment is handled automatically by the workflow main_bdsagroup17chirpremotedb.yml. Deploys to https://bdsagroup17chirpremotedb-dhg0b9fpaya0afa0.swedencentral-01.azurewebsites.net/

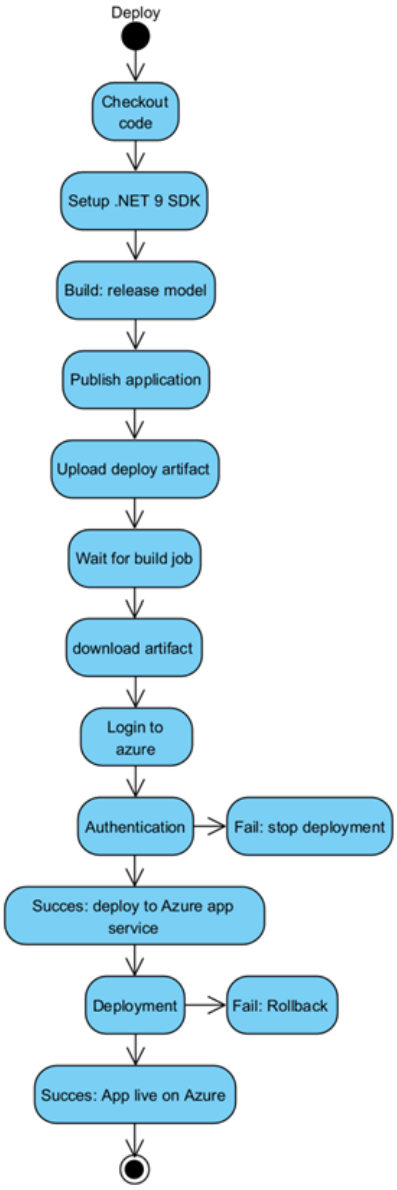

Figure 14: Deploy workflow

2.2 Team work

2.2.1 Project board

	Title	Assignees	Status
1	(week 5) Add initial tests for Razor app #15	A-thenaaa and Lo...	Done
2	Refactor data persistence with a secondary library project #4		Done
3	Automate build, test, and publishing with GitHub Workflows (03_1a) #5		Done
4	In order to do good in class as a student, I can write an issue #1		Done
5	Use an external library to handle csv files #3		Done
6	Enhancement: Move code for interface into class #2		Done
7	(week 5) modify GetCheeps() in CheepService.cs so it uses cheeps.db #11		Done
8	(week 5) Implement DBFacade and SQLite connection #12		Done
9	(week 5) Refactor CheepService to use DBFacade #13		Done
10	(week 5) Add pagination to timelines #14		Done
11	(week 5) Move CLI to chirp_cli branch #16		Done
12	Allow a GitHub action to create releases	nschiang	Done
13	(week 5) Update GitHub Actions for Razor app #17	GetFedBread	Done
14	(week 5) Introduce pull request workflow and code reviews #18		Done
15	As a developer, I need to add login to Chirp! so users can authenticate via ASP.NET Co... #42		Done
16	Share messages on Chirp! #46		Done
17	Forget me functionality #68		Done
18	View personal information #67		Done
19	As a developer, I need to constrain cheeps to a maximum length of 160 characters so... #33		Done
20	As an authenticated user, I want to unfollow another user so that I stop seeing their ... #59		Done
21	View followed users #74		Done
22	Saved cheeps #75		Done
23	Reload on OAuth login #83		Todo
24	Ensure delete logic is exclusively handled in the Service layer #86		Done
25	Reconsider null vs exception handling in AuthorRepository #90		Todo

Figure 15: Screenshot of the GitHub Project board before hand-in.

We used a GitHub Project board to track and manage all development tasks throughout the project. Each task was created as a GitHub Issue was moved across the board as work progressed. The typical workflow for implementing a feature was:

1. A new Issue is created describing the task and acceptance criteria.
2. The Issue is moved to **In progress** when development begins.
3. The feature is implemented on a separate branch.
4. A Pull Request is opened against the `main` branch.
5. Automated CI workflows run build and test pipelines.
6. After review and successful checks, the Pull Request is merged into `main`.
7. The Issue is moved to **Done** on the project board. We used a GitHub Project board to track and manage all development tasks throughout the project. Each task was created as a GitHub Issue and moved across the board as work progressed.

Most tasks are marked as **Done** at the time of hand-in. The remaining unresolved

tasks are the following:

- **AuthorRepository behavior**:
  The repository currently returns `null` when an author is not found instead of throwing an exception.
  It is not yet decided whether this represents the expected control flow or an exceptional case.
  Additional tests are required to validate the chosen behavior and ensure all callers handle it safely.
- **OAuth login refresh issue**:
  After authenticating via GitHub OAuth the user currently needs to reload the page before the login state takes effect.
  The expected behavior is that authentication is reflected immediately without a manual reload.

2.2.2 Development flow

Early in the development of Chirp it was decided that the whole group would work collectively on all the tasks. We felt that many of the tasks depended on each other, and since we had enough time to complete most tasks every week it was best for everyone if we as a group did everything together. This is shown when commiting new code to the project by all present team members being co-authored. As a consequence of this way of working, most code reviews on the pull requests are sparse, as we all watched the code being written and pitched in, meaning there wasn't much need for further communication in regards to the code.

When starting work on a new set of weekly tasks we began by making issues on Github for each of the tasks, which we would then work through during the week. On account of our way of doing teamwork, where everyone was involved most of the time, pull requests would rarely not be approved, but this option is still illustrated in the diagram below.

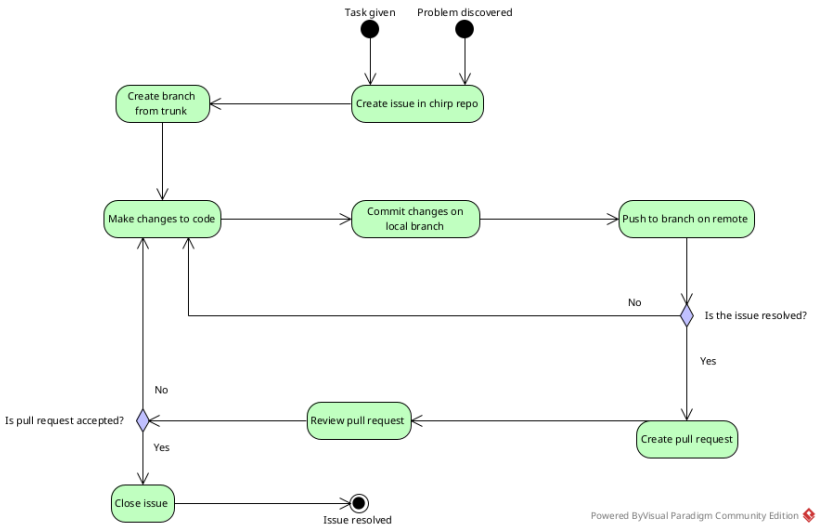

Figure 16: Flow diagram from issue creation to resolution

2.3 How to make *Chirp!* work locally

2.3.1 Prerequirements:

- .NET 9 SDK installed
- Git

2.3.2 Steps

1. Clone the repository: `git clone https://github.com/ITU-BDSA2025-GROUP-17/Chirp`
2. After the cloning the project, go to the project: `cd Chirp`
3. Restore dependencies: `dotnet restore src/Chirp.Web/Chirp.Web.csproj`
4. Run the application

If you wish to access the application with Github OAuth (This also requires secrets to be set up):

```
dotnet run --project src/Chirp.Web/Chirp.Web.csproj --launch-profile
https
```

else:

```
dotnet run --project src/Chirp.Web/Chirp.Web.csproj
```

1. Access the application
    - Open browser and navigate to: http://localhost:7273 or https://localhost:7273 if using https launch profile

- You should see the Chirp public timeline with seeded cheeps

Notes:

- The application can be run locally without configuring GitHub authentication.
- GitHub login will not work locally unless user secrets are configured.
- This does not affect core functionality such as browsing cheeps or local authentication.

2.3.2.1 GitHub authentication

GitHub authentication relies on OAuth secrets which are not stored in the repository. To enable GitHub login locally, user secrets must be configured manually:

```
dotnet user-secrets set "Authentication:GitHub:ClientId"
   "<your-client-id>" --project src/Chirp.Web
dotnet user-secrets set "Authentication:GitHub:ClientSecret"
   "<your-client-secret>" --project src/Chirp.Web
```

These secrets are provided via GitHub OAuth and are intentionally not included in the repository. In the deployed Azure environment, the secrets are configured securely using Azure App Service settings.

2.4 How to run test suite locally

2.4.0.1 Unit tests:

```
dotnet test test/Chirp.Repositories.Tests
```

2.4.0.2 Integration tests:

```
dotnet test test/Chirp.IntegrationTests
```

2.4.0.3 End-to-end tests (requires Playwright):

Windows (using powershell)

```
cd test/ChirpEndToEndTests
dotnet build
pwsh bin/Debug/net9.0/playwright.ps1 install
dotnet test
```

Mac/Linux

```
cd test/ChirpEndToEndTests
dotnet build
./bin/Debug/net9.0/playwright.sh install
dotnet test
```

3 Ethics

3.1 License

The Chirp! Project is released under the MIT license. This is a permissive open-source license that allows others to use, modify, distribute and build upon the software with very few restrictions. The only requirement is that the original copyright notice and license text are included in any copies or substantial portions of the software. The software is provided "as is", with any warranty, which means the developers are not liable for potential issues arising from its use.

3.2 LLMs, ChatGPT, CoPilot, and others

During development of the project, we used several Large Language Models (LLMs), including ChatGPT, GitHub Copilot and Claude.

ChatGPT was used as a support tool for understanding code and concepts. Use cases included clarifying the meaning of specific lines of code, helping with how to write smaller code lines and explaining why certain methods or implementations caused issues, replacing the need to searching through documentation or Stack Overflow in most cases. This helped save time and allowed us to focus more on understanding and writing the code ourselves. ChatGPT was also used in a theoretical manner to discuss architectural and conceptual decisions before implementation, ensuring that we had a solid understanding before writing code. Additionally, it was used to help rephrase or improve issue descriptions and parts of the written report.

Claude was used as a kind of teaching assistant after the code had already been reviewed within the group and there was still uncertainty about the solution. We intentionally used it for guidance rather than complete answers. It was also helpful when working with tests, especially for interpreting and applying the guidelines from the course literature when implementing integration tests. In a few cases, small code snippets were used as inspiration rather than finished solutions.

GitHub Copilot was used passively during development, mainly by automatically generating commit or pull request messages, some of which were accepted.

Overall LLMs were used as supportive tools rather than sources of complete solutions. They helped speed up development, reduce time spent on searching for information and improve understanding, while the main implementation and problem solving were still thought out by the group.